

PROYECTO DE TESIS

ManyHands: orquestación verificable de agentes para desarrollo de software

ManyHands transforma un objetivo de desarrollo en un grafo jerárquico de tareas, las ejecuta con agentes aislados bajo contratos explícitos y recompone un resultado integrado y verificable.

Francisco Clarke Ingeniería en Sistemas de Información

PROBLEMA Y MOTIVACIÓN

Una funcionalidad grande no es una sola tarea

Ejemplo: **agregar recuperación segura de contraseña a un portal existente** cruza API, dominio, persistencia, interfaz y tests. Tratarla como una única tarea agentic lineal puede encontrar límites:

01	Tamaño y heterogeneidad del objetivo; el contexto necesario no entra de una vez	04	Interferencia sobre el repositorio cuando hay trabajo concurrente
02	Dependencias entre cambios que la ejecución lineal deja implícitas	05	Validación parcial: cada pieza puede «pasar» sin que el conjunto funcione
03	Dificultad para sostener una planificación coherente durante toda la tarea	06	Integración de resultados relacionados al final, sin estructura que la guíe

Planificación, dependencias e integración dejan de ser implícitas: **pasan a ser problemas explícitos que el sistema debe representar y resolver.**

Descomposición e integración guiadas por un DAG

PREGUNTA PRINCIPAL

¿Puede una arquitectura basada en **descomposición jerárquica, ejecución coordinada e integración bottom-up** hacer abordables funcionalidades difíciles de resolver como una única tarea agentic lineal?

HIPÓTESIS DE INGENIERÍA

Representar el trabajo como un **DAG jerárquico con contratos explícitos** permite dividir una funcionalidad grande en unidades acotadas, identificar dependencias, habilitar paralelismo seguro y recomponer los resultados de manera controlada.

SEGUNDA PREGUNTA · GRANULARIDAD (EXPLORATORIA)

Demasiado gruesa: tareas grandes, heterogéneas y difíciles de validar. Demasiado fina: más contratos, dependencias, coordinación e integración. **¿Es posible estimar una granularidad adecuada por tarea** según su complejidad, alcance, interfaces, estructura del repositorio y costo de coordinación?

Motiva el modo «auto» del decomposer. La política adaptativa completa y su evaluación comparativa son trabajo futuro — no un resultado.

Un run planificado, ejecutado, validado e integrado

Objetivo general: diseñar e implementar un sistema que transforme un objetivo de desarrollo en un run **planificado, ejecutado, validado, integrado y supervisable**.

OBJETIVOS ESPECÍFICOS	ALCANCE DE LA IMPLEMENTACIÓN
• Planificación basada en el repositorio • Contratos y grafo ejecutable • Ejecución aislada y concurrente • Validación e integración • Persistencia, supervisión y recuperación	• Sistema local-first, single-host • Ejecución mediante agentes CLI • Aislamiento con Git worktrees • Persistencia durable local (JSONL) • Interfaz web para supervisión

Sin infraestructura cloud ni ejecución distribuida: quedan fuera del alcance implementado. La política adaptativa completa de granularidad es trabajo futuro.

Incremental, con decisiones y evidencia registradas

Desarrollo incremental por vertical slices

Migración controlada desde la arquitectura anterior, sin big-bang.

Decisiones registradas en ADRs

Contexto, alternativas y consecuencias; DECISIONS.md fija lo vigente.

Tests junto con cada cambio de comportamiento

Unit, integración, boundaries y E2E de dominio; Git y filesystem reales, no mocks.

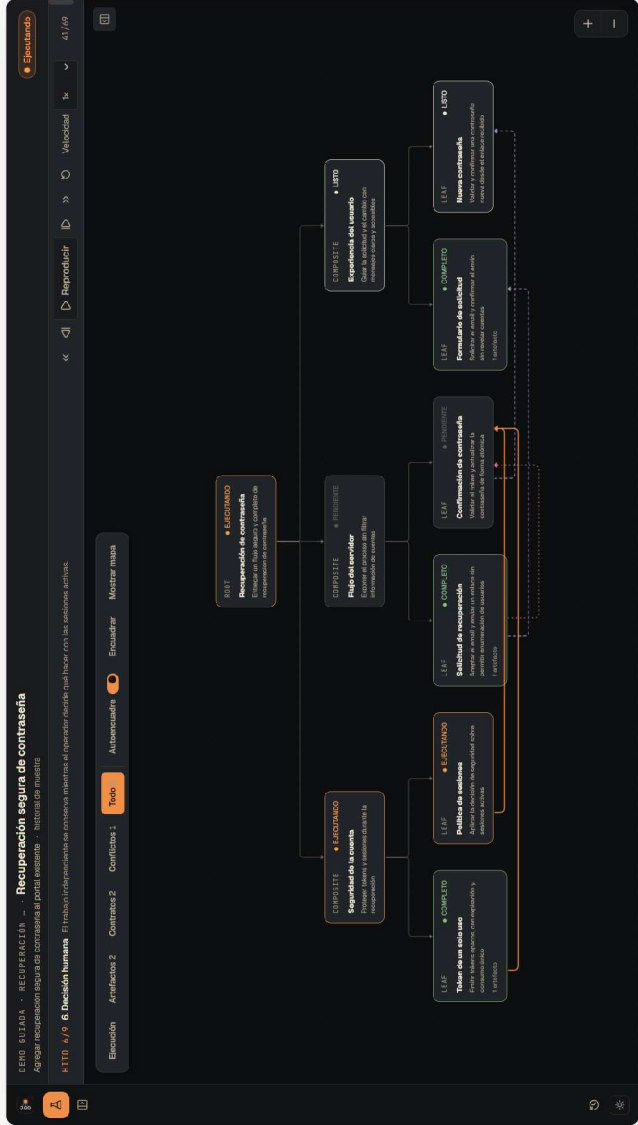
Auditorías de la ruta productiva y smokes manuales

Runs reales inspeccionados por journal; limitaciones documentadas.

Tres niveles de certeza, siempre diferenciados: **verificado por tests** · **observado en smoke** · **trabajo futuro**. Esta disciplina permitió revertir decisiones insuficientes sin perder el sistema.

VISIÓN GENERAL

Flujo completo de un run



UI actual: run en ejecución con hito de decisión humana, estados por nodo y vistas de artefactos, contratos y conflictos. Evidencia por etapa: código + tests; E2E de dominio hasta result_ready; smoke productivo auditado hasta needs_approval (2026-07-18).

El planner propone; el Graph Compiler produce el plan ejecutable

Goal + RepositorySnapshot (grounding: estructura real del repo)
WorkBreakdown propuesto por el planner ← salida del modelo
Validación con schema Zod (inválido ⇒ no entra al dominio)
Graph Compiler: identidad, nodos leaf / composite, contratos
Critics deterministas: completitud, atomicidad, DAG
GraphRevision inmutable, lista para aprobación

El modelo no gobierna lifecycle, scheduling ni persistencia; una ambigüedad legítima se vuelve decisión humana durable.

Evidencia: graph-compiler.test.ts · contracts-v2.test.ts · decomposer-work-breakdown.test.ts · scheduler-readiness-v2.test.ts

RELACIÓN TIPADA	PROBLEMA QUE RESUELVE
parentId	Ownership : un único dueño de integración por nodo
ArtifactRequirement	Disponibilidad : qué output material debe existir antes; ordena el scheduling
SeamBinding	Compatibilidad : siblings en paralelo contra una interfaz congelada; no impone orden
ConflictConstraint	Riesgo : restringe la wave sin inventar una dependencia funcional

Ejecución aislada y adopción de resultados

- 1 El scheduler calcula **readiness** y despacha waves: paralelismo condicionado por dependencias y riesgo
- 2 ExecutionBase exacta: commit base + solo los artifacts declarados; un Git worktree por intento
- 3 InputFingerprint: hash de todas las entradas del intento — su identidad
- 4 Scope **deny-wins**; el orquestador inspecciona el diff real y crea el **candidate commit**
- 5 Entradas cambiadas ⇒ resultado **stale**, **se rechaza**; fuera de scope ⇒ intento descartado

```
if (attempt.inputFingerprint !==
    input.currentFingerprint) {
  return { eligible: false, reason,
    event: { type: "attempt.stale",
      payload: { attemptId, nodeId,
        ... } } };
}
```

run-coordinator/src/domain/attempt.ts

Invariante: un resultado calculado con entradas anteriores nunca se adopta. Tests: `input-fingerprint · attempt-adoption · execution-core-worktree · execution-core-scope`

AgentExecutor (perfiles: Claude Code CLI · Codex CLI) — el agente propone cambios en su worktree; qué cambió lo dice el Git diff, no su stdout.

Integración bottom-up y evidencia sobre el commit exacto

VALIDACIÓN E INTEGRACIÓN

- Validación en un entorno Git temporal limpio sobre el **SHA exacto** del candidato
- EvidenceMatrix: un veredicto por criterio de aceptación
- Outputs adoptados → **Artifact Registry**
- Cada composite integra bottom-up; su IntegrationManifest declara qué outputs hijos aplicó
- Aprobación → DeliveryReceipt; publicación idempotente contra un target sin cambios

ESTADO DURABLE Y RECUPERACIÓN POR CAUSA

- Journal de eventos **append-only**; el estado se reconstruye por replay
- CAS de secuencia, leases y fencing tokens rechazan writers viejos y callbacks tardíos
- Transitorio ⇒ nuevo intento · código/tests ⇒ una reparación con diagnóstico
- Contrato incorrecto ⇒ enmienda · **stale** ⇒ **descarte**, nunca adopción
- Decisiones humanas durables que bloquean solo su alcance

Tecnologías atadas a responsabilidades concretas

ZOD: LOS DATOS NO CONFIABLES SE PARSEAN EN EL LÍMITE

```
function event(type, payload) {
  return z.object({ ...BaseEventShape,
    type: z.literal(type),
    payload }).strict();
}

export const RunEventSchema =
  z.discriminatedUnion("type", [
    event("run.created", ...), ... ]]);
```

run-coordinator/src/domain/events.ts

HTTP, salida de modelos, SSE y disco se parsean al cruzar el límite · z.infer alinea schema y tipo · superRefine para invariantes cross-field. **Límite:** Zod valida forma; lifecycle, freshness y adopción son reglas del dominio.

TypeScript	Todo el monorepo; tipos inferidos desde schemas runtime
Next.js	Transporte HTTP y composition root; el dominio no lo importa
React + React Flow	Adapter visual del grafo y supervisión del run
Git + simple-git	Worktrees por intento, candidate commits, validación exacta
Claude Code · Codex CLI	Perfiles de AgentExecutor; providers intercambiables
Vitest	Unit, integración con Git real, boundaries y E2E de dominio
JSON / JSONL	Journal append-only con checksums; snapshots descartables

LangGraph: usado en la arquitectura anterior; al detectar múltiples autoridades de estado trasladé el lifecycle a RunCoordinator. Hoy sin imports productivos — evolución completa en respaldo.

Resultados y límites actuales

RESULTADOS, POR NIVEL DE EVIDENCIA	QUÉ RESOLVIÓ LA ARQUITECTURA	LÍMITES ACTUALES
• Verificado por tests: recorrido completo de dominio hasta result_ready (E2E); 915 tests en 156 archivos (auditoría 18/07/2026) • Observado en smoke: planning greenfield, decisiones durables y replan hasta needs_approval • Pendiente: smoke productivo end-to-end hasta delivery	• Una sola autoridad de lifecycle, testeable sin frameworks • Adopción segura de resultados concurrentes (fingerprints) • Evidencia sobre el commit entregado, no sobre el relato del agente • Recuperación por causa, sin retry universal	• Local-first y single-host • Worktrees, no sandbox de seguridad • Streaming fuerte solo con Claude Code CLI • Sin multi-tenancy ni distribución • Granularidad adaptativa: línea futura

Estas son las decisiones técnicas que implementé, las garantías que pude verificar y los límites que todavía conserva el sistema.

Fragmentos de código y tests

```
const canonical = { ...input,
  contractRevisions:
    [...].sort(compareRef),
  consumedArtifacts:
    [...].sort(compareRef) };
return `sha256:${createHash(
  "sha256")...digest("hex")}`;
```

domain/fingerprint.ts — orden canónico ⇒
identidad estable · input-fingerprint.test.ts

```
// fold ANTES de persistir
foldRun([...current,
  ...provisional]);
const appended = await
  this.ports.events.append(
    runId, expectedSequence,
    inputs);
```

coordinator.ts — una transición ilegal no
llega al journal · run-events-replay.test.ts

```
if (target.branch !==
  approval.targetBranch
  || target.head !==
    approval.targetHead) {
  throw new Error("The delivery
    target changed...; nothing
    was published.");
}
```

delivery/publisher.ts — se publica
exactamente lo aprobado · delivery-
state-machine.test.ts

Rutas completas bajo packages/run-coordinator/src/ y packages/execution-core/src/

Evolución de LangGraph y trade-offs

uso	StateGraph y checkpoints participaban del flujo productivo de la arquitectura anterior	
problema	Múltiples autoridades de estado: checkpoints, RunRecord, UI y journal podían divergir	
decisión	Lifecycle trasladado a RunCoordinator independiente de frameworks; StateGraphs productivos retirados (c5a4f99)	
aprendizaje	Los checkpoints no sustituyen un journal de dominio; la autoridad del lifecycle debe ser única; el framework queda como posible adapter si no duplica estado	

Lifecycle propio vs. StateGraph

Costo: más código de dominio. **Ganancia:** una sola autoridad de estado, testeable sin el framework.

Journal de eventos vs. checkpoints

Costo: reducir y replay propios. **Ganancia:** historia auditable; snapshots descartables; recovery explicable.

Matriz de recuperación y limitaciones operativas

CAUSA	RESPUESTA	LIMITACIONES OPERATIVAS ACTUALES
transitorio	reintento acotado (nuevo attempt)	• Latencia del planner antes del primer nodo, todavía sin medición visible • Streaming progresivo fuerte solo con Claude Code CLI • Smoke productivo end-to-end hasta delivery pendiente • Local-first y single-host; worktrees locales, sin VM ni container de seguridad • Sin multi-tenancy ni infraestructura distribuida
autenticación / binario / entorno	suspender recurso y pedir corrección	
código o test local	una reparación en el mismo worktree	
contrato / descomposición	enmienda / replan local	
dependencia no declarada	registrar y enmendar el grafo	
scope o commit inesperado	descartar el intento	
integración	una reparación semántica; luego decisión	
infraestructura compartida	suspender solo el trabajo afectado	